services\data\data_processor.py

```
import os
import json
import gzip
from typing import List, Dict, Any, Optional, Tuple
from datetime import datetime
import config
from database.db_manager import db_manager
from services.cache.cache_manager import cache_manager
import gc

class DataProcessor:
"""Process and extract insights from historical NVD data with lazy loading and database
caching"""
def __init__(self):
self.historical_dir = config.NVD_HISTORICAL_DIR
self.processed_dir = config.NVD_PROCESSED_DIR

# CRITICAL: In-memory cache for SINGLE year only
self._year_cache = {}
self._max_cache_size = 1  # Only cache 1 year at a time
self._years_available = set()  # Track which years have files

print(f"[Historical] DataProcessor initialized")
print(f"[Historical] Local directory: {self.historical_dir}")

# Check what files are available
self._check_available_files()

def _check_available_files(self):
"""Check what historical files are available"""
self._years_available.clear()

if not os.path.exists(self.historical_dir):
print(f"[Historical] Directory does not exist: {self.historical_dir}")
print(f"[Historical] Creating directory...")
os.makedirs(self.historical_dir, exist_ok=True)
return

files = os.listdir(self.historical_dir)
cve_files = [f for f in files if f.startswith(('CVE-', 'nvdcve-'))]

if cve_files:
print(f"[Historical] Found {len(cve_files)} files: {cve_files}")

# Extract years from filenames
for file in cve_files:
year = None

# Try different filename patterns
if file.startswith("CVE-") and "." in file:
year_str = file[4:file.find(".")]
if year_str.isdigit() and len(year_str) == 4:
year = int(year_str)

elif file.startswith("nvdcve-") and "-" in file:
parts = file.split("-")
if len(parts) >= 3 and parts[2].isdigit() and len(parts[2]) == 4:
year = int(parts[2])

if year and 2000 <= year <= 2050:  # Sanity check for valid years
self._years_available.add(year)

print(f"[Historical] Available years: {sorted(self._years_available)}")
else:
print(f"[Historical] WARNING: No files found in {self.historical_dir}")
print(f"[Historical] Please add CVE-XXXX.json or CVE-XXXX.json.gz files to this
directory")

def get_available_years(self) -> List[int]:
"""Get list of available years"""
if not self._years_available:
self._check_available_files()
```

```
return sorted(self._years_available, reverse=True)

def get_year_data(self, year: int) -> List[Dict]:
"""Load CVEs from a specific year file - WITH DATABASE CACHING"""
year_str = str(year)

# First check if we already have this data in the database
if db_manager.use_database:
cves = db_manager.get_cves_by_filter(year=year_str)
if cves:
print(f"[Historical] Retrieved {len(cves)} CVEs for {year} from database")
return cves

# Fall back to memory cache
if year_str in self._year_cache:
print(f"[Historical] Using cached data for {year}")
return self._year_cache[year_str]

# Load from local file
data = self._load_year_file(year)

# If database is enabled, store the data
if db_manager.use_database and data:
print(f"[Historical] Saving {len(data)} CVEs for {year} to database")
db_manager.save_cves_batch(data)

# Cache management - only keep 1 year in memory
if len(self._year_cache) >= self._max_cache_size:
# Clear oldest cache entry
oldest_year = next(iter(self._year_cache))
print(f"[Historical] Evicting {oldest_year} from cache to save memory")
del self._year_cache[oldest_year]
# Force garbage collection to free memory
gc.collect()

# Only store in memory cache if database is not available
if not db_manager.use_database:
self._year_cache[year_str] = data
print(f"[Historical] Cached {len(data)} CVEs for {year} in memory")
else:
print(f"[Historical] Not caching in memory as database is available")
# Don't even keep temporary memory cache if DB is enabled
self._year_cache.clear()

return data

def _load_year_file(self, year: int) -> List[Dict]:
"""Load CVEs from a specific year file - LOCAL FILES ONLY
This method supports both compressed (.json.gz) and uncompressed (.json) files.
"""
try:
# Try different file patterns - INCLUDING GZIPPED FILES
possible_filenames = [
f"CVE-{year}.json.gz",  # Try gzipped first (most common)
f"CVE-{year}.json",
f"nvdcve-1.1-{year}.json.gz",
f"nvdcve-1.1-{year}.json",
f"nvdcve-2.0-{year}.json.gz",
f"nvdcve-2.0-{year}.json",
]

# Check if directory exists
if not os.path.exists(self.historical_dir):
print(f"[Historical] Directory does not exist: {self.historical_dir}")
return []

# Find the first available file
target_file = None
filename_used = None
is_gzipped = False

for filename in possible_filenames:
file_path = os.path.join(self.historical_dir, filename)
if os.path.exists(file_path):
target_file = file_path
filename_used = filename
is_gzipped = filename.endswith('.gz')
```

```
print(f"[Historical] Found file: {filename} (gzipped={is_gzipped})")
break

if not target_file:
print(f"[Historical] No file found for {year}. Tried: {possible_filenames}")
print(f"[Historical] Available files: {os.listdir(self.historical_dir)}")
return []

print(f"[Historical] Loading file: {target_file}")

# Check file size
file_size = os.path.getsize(target_file)
print(f"[Historical] File size: {file_size:,} bytes ({file_size/1024/1024:.2f} MB)")

# Load the file - HANDLE BOTH GZIPPED AND UNCOMPRESSED
data = None
if is_gzipped:
print(f"[Historical] Opening gzipped file...")
with gzip.open(target_file, 'rt', encoding='utf-8') as f:
data = json.load(f)
print(f"[Historical] Successfully loaded gzipped JSON file for {year}")
else:
print(f"[Historical] Opening uncompressed file...")
with open(target_file, 'r', encoding='utf-8') as f:
data = json.load(f)
print(f"[Historical] Successfully loaded JSON file for {year}")

# Process CVEs based on detected format
cves = []

# Try JSON 2.0 format first (newer format)
vulnerabilities = data.get("vulnerabilities", [])
print(f"[Historical] DEBUG: vulnerabilities field exists: {vulnerabilities is not
None}")
print(f"[Historical] DEBUG: vulnerabilities length: {len(vulnerabilities) if
vulnerabilities else 0}")

if vulnerabilities:
print(f"[Historical] Found {len(vulnerabilities)} vulnerabilities in JSON 2.0 format")

# For large vulnerability datasets, process in batches to manage memory
processed_count = 0
failed_count = 0
batch_size = 5000

for i in range(0, len(vulnerabilities), batch_size):
batch = vulnerabilities[i:i+batch_size]
print(f"[Historical] Processing batch {i//batch_size + 1} ({len(batch)} items)")

for item in batch:
processed = self._process_json_2_0_cve(item)
if processed:
cves.append(processed)
processed_count += 1
else:
failed_count += 1

# Clear batch to free memory
del batch
gc.collect()

print(f"[Historical] DEBUG: Processed {processed_count} CVEs, {failed_count} failed")

else:
# Try JSON 1.1 format (legacy format)
cve_items = data.get("CVE_Items", [])
print(f"[Historical] DEBUG: CVE_Items field exists: {cve_items is not None}")
print(f"[Historical] DEBUG: CVE_Items length: {len(cve_items) if cve_items else 0}")

if cve_items:
print(f"[Historical] Found {len(cve_items)} CVE_Items in JSON 1.1 format")

# Process in batches
processed_count = 0
failed_count = 0
batch_size = 5000
```

```
for i in range(0, len(cve_items), batch_size):
batch = cve_items[i:i+batch_size]
print(f"[Historical] Processing batch {i//batch_size + 1} ({len(batch)} items)")

for item in batch:
processed = self._process_json_1_1_cve(item)
if processed:
cves.append(processed)
processed_count += 1
else:
failed_count += 1

# Clear batch to free memory
del batch
gc.collect()

print(f"[Historical] DEBUG: Processed {processed_count} CVEs, {failed_count} failed")

else:
print(f"[Historical] ERROR: No recognized data format found")

# Clear the raw data to free memory
del data
gc.collect()

print(f"[Historical] Successfully processed {len(cves)} CVEs for {year}")

return cves

except Exception as e:
print(f"[Historical] ERROR loading {year}: {e}")
import traceback
traceback.print_exc()
return []

def _process_json_2_0_cve(self, item: Dict) -> Optional[Dict]:
"""Process JSON 2.0 format CVE - MEMORY OPTIMIZED"""
try:
cve_data = item.get("cve", {})
if not cve_data:
return None

cve_id = cve_data.get("id", "")
if not cve_id:
return None

# Extract English description
description = ""
for desc in cve_data.get("descriptions", []):
if desc.get("lang") == "en":
description = desc.get("value", "")
break

# Extract severity
severity = "UNKNOWN"
cvss_score = None

metrics = cve_data.get("metrics", {})
for version in ["cvssMetricV31", "cvssMetricV30", "cvssMetricV2"]:
if version in metrics and metrics[version]:
try:
metric = metrics[version][0]
cvss_data = metric.get("cvssData", {})
potential_severity = cvss_data.get("baseSeverity", "")
if potential_severity and potential_severity.upper() != "UNKNOWN":
severity = potential_severity.upper()
cvss_score = cvss_data.get("baseScore")
break
except (IndexError, KeyError):
continue

# Extract CWE
cwe = None
for weakness in cve_data.get("weaknesses", []):
for desc in weakness.get("description", []):
if desc.get("lang") == "en":
value = desc.get("value", "")
```

```
if value.startswith("CWE"):
cwe = value
break
if cwe:
break

# Extract limited references to save memory
references = []
for ref_data in cve_data.get("references", [])[:3]:  # Limit to first 3 references
ref_url = ref_data.get("url", "")
if ref_url:
references.append({"url": ref_url})

return {
"ID": cve_id,
"Description": description,
"Severity": severity,
"CWE": cwe,
"Published": cve_data.get("published", ""),
"lastModified": cve_data.get("lastModified", ""),
"References": references,
"Products": [],  # Empty to save memory
"CVSS_Score": cvss_score,
"metrics": {}  # Empty to save memory
}
except Exception as e:
return None

def _process_json_1_1_cve(self, item: Dict) -> Optional[Dict]:
"""Process JSON 1.1 format CVE - MEMORY OPTIMIZED"""
try:
cve_data = item.get("cve", {})
if not cve_data:
return None

cve_id = cve_data.get("CVE_data_meta", {}).get("ID", "")
if not cve_id:
return None

# Extract English description
description = ""
descriptions = cve_data.get("description", {}).get("description_data", [])
for desc in descriptions:
if desc.get("lang") == "en":
description = desc.get("value", "")
break

# Extract severity
severity = "UNKNOWN"
cvss_score = None

impact = item.get("impact", {})
if "baseMetricV3" in impact:
severity = impact["baseMetricV3"].get("cvssV3", {}).get("baseSeverity",
"UNKNOWN").upper()
cvss_score = impact["baseMetricV3"].get("cvssV3", {}).get("baseScore")
elif "baseMetricV2" in impact:
score = impact["baseMetricV2"].get("cvssV2", {}).get("baseScore", 0)
cvss_score = score
if score >= 7.0:
severity = "HIGH"
elif score >= 4.0:
severity = "MEDIUM"
else:
severity = "LOW"

# Extract CWE
cwe = None
problem_types = cve_data.get("problemtype", {}).get("problemtype_data", [])
for problem_type in problem_types:
for desc in problem_type.get("description", []):
if desc.get("lang") == "en":
value = desc.get("value", "")
if value.startswith("CWE"):
cwe = value
break
if cwe:
```

```
break

# Extract limited references to save memory
references = []
ref_data = cve_data.get("references", {}).get("reference_data", [])
for ref in ref_data[:3]:  # Limit to first 3 references
ref_url = ref.get("url", "")
if ref_url:
references.append({"url": ref_url})

return {
"ID": cve_id,
"Description": description,
"Severity": severity,
"CWE": cwe,
"Published": cve_data.get("publishedDate", ""),
"lastModified": cve_data.get("lastModifiedDate", ""),
"References": references,
"Products": [],  # Empty to save memory
"CVSS_Score": cvss_score,
"metrics": {}  # Empty to save memory
}
except Exception as e:
return None

def get_year_cve_counts(self) -> Dict[int, int]:
"""Get counts of CVEs for each year in database - NEW OPTIMIZED METHOD"""
# First try to get from database
if db_manager.use_database:
stats = db_manager.get_summary_stats('yearly_cve_counts')
if stats and 'data' in stats:
return stats['data']

# Fall back to checking files if database doesn't have the info
result = {}

# Use available years from earlier check
if not self._years_available:
self._check_available_files()

for year in self._years_available:
# Instead of loading all data, just check if the file exists and get its size
possible_filenames = [
f"CVE-{year}.json.gz",
f"CVE-{year}.json",
f"nvdcve-1.1-{year}.json.gz",
f"nvdcve-1.1-{year}.json"
]

for filename in possible_filenames:
file_path = os.path.join(self.historical_dir, filename)
if os.path.exists(file_path):
file_size = os.path.getsize(file_path)
# Estimate count based on file size (rough approximation)
estimated_count = int(file_size / 500)  # Assume average 500 bytes per CVE
result[year] = estimated_count
break

# Save to database if available
if db_manager.use_database:
db_manager.save_summary_stats('yearly_cve_counts', len(result), result)

return result

def get_filtered_cves_for_year(self, year: int, month: Optional[int] = None, day:
Optional[int] = None) -> List[Dict]:
"""Get CVEs for a specific year with optional month/day filtering - OPTIMIZED"""
# Try database first
if db_manager.use_database:
year_str = str(year)
month_str = str(month).zfill(2) if month is not None else None
day_str = str(day).zfill(2) if day is not None else None

cves = db_manager.get_cves_by_filter(
year=year_str,
month=month_str,
day=day_str
```

```
)

if cves:
print(f"[Historical] Retrieved filtered CVEs from database: {len(cves)} CVEs for
{year}-{month_str or '*'}-{day_str or '*'}")
return cves

# Fall back to loading and filtering from files
all_cves = self.get_year_data(year)

if not month and not day:
return all_cves

filtered_cves = []
for cve in all_cves:
published = cve.get('Published', '')
if not published:
continue

try:
# Handle different date formats
if 'T' in published:
# ISO format with time
dt = datetime.fromisoformat(published.replace('Z', '+00:00'))
else:
# Date only format
dt = datetime.strptime(published[:10], '%Y-%m-%d')

# Apply month filter
if month is not None and dt.month != month:
continue

# Apply day filter
if day is not None and dt.day != day:
continue

filtered_cves.append(cve)
except:
# Skip entries with invalid date format
continue

print(f"[Historical] Filtered {len(filtered_cves)} CVEs for {year}-{month or '*'}-{day
or '*'}")
return filtered_cves

def calculate_vulnerabilities_timeline(self, years=1) -> Dict[str, Any]:
"""Calculate vulnerabilities over time for the specified number of years - DATABASE
OPTIMIZED"""
# Try to get from database cache first
cached_timeline = cache_manager.get_timeline_data(years)
if cached_timeline and cached_timeline.get('labels'):
print(f"[Historical] Using cached timeline data for {years} years")
return cached_timeline

# Determine years to include
current_year = datetime.now().year
year_range = list(range(current_year - years + 1, current_year + 1))
print(f"[Vulnerabilities Over Time] Loading years: {year_range}")

result = {
'labels': [],
'values': [],
'total_cves': 0,
'months_covered': 0,
'raw_data': {}
}

# Get data for each year
for year in year_range:
# Only process years that have data
if year not in self._years_available:
continue

# Use our optimized method to get data for this year
cves = self.get_year_data(year)
print(f"[Vulnerabilities Over Time] Processing {year}: {len(cves)} CVEs")
```

```
# Group by month
month_counts = {}
for cve in cves:
published = cve.get('Published', '')
if not published:
continue

try:
# Handle different date formats
if 'T' in published:
dt = datetime.fromisoformat(published.replace('Z', '+00:00'))
else:
dt = datetime.strptime(published[:10], '%Y-%m-%d')

month_key = f"{dt.year}-{dt.month:02d}"
month_counts[month_key] = month_counts.get(month_key, 0) + 1
except:
continue

# Clear year data to save memory
if year_str := str(year) in self._year_cache:
print(f"[Vulnerabilities Over Time] Clearing cache for {year} to save memory")
del self._year_cache[year_str]
gc.collect()

# Add to results
for month_key, count in sorted(month_counts.items()):
result['labels'].append(month_key)
result['values'].append(count)
result['raw_data'][month_key] = count
result['total_cves'] += count

result['months_covered'] = len(result['labels'])

# Cache the results in the database
cache_manager.set_timeline_data(result)

return result

def clear_cache(self):
"""Clear in-memory cache"""
self._year_cache.clear()
print("[Historical] In-memory cache cleared")

# Global historical loader instance
historical_loader = DataProcessor()
```